

Full Summary -- All Contribution Areas

Author: Yasithzz | 2026

05 -- Full Summary of All Contribution Areas

LifeMate Project -- Viva / Presentation Documentation

What is LifeMate?

LifeMate is an AI-powered personal wellness companion application. It addresses a widely prevalent modern problem -- chronic stress -- by combining daily lifestyle tracking, machine learning prediction, and intelligent schedule generation into a seamless user experience.

Technology Stack:

- * Frontend: React + TailwindCSS
- * Backend: Spring Boot + MongoDB Atlas
- * ML Service: Python FastAPI + scikit-learn (RandomForestClassifier)

Four Contribution Areas -- Yasithzz

1. Questionnaire Module

What it does: Collects eight daily lifestyle data points from the user through an interactive form and sends them to the ML model for stress prediction.

The 8 Lifestyle Inputs

Input	Description	Range
Mood	Emotional state today	1 (Stressed) -> 5 (Great)
Workload	Perceived work pressure	1 (Very Light) -> 5 (Very Heavy)
Sleep Hours	Hours slept last night	0-12
Energy Level	Physical energy	1 (Exhausted) -> 5 (Very Energized)
Social Interaction	Social engagement	1 (None) -> 5 (Very Social)
Exercise Done	Whether exercise was done	0 = No, 1 = Yes
Screen Time	Hours on phone/PC	0-14
Water Cups	Cups of water consumed	0-16

Flow

- * User fills the daily check-in form on LifestylePage.jsx.
- * Submit -> backend validates -> ML Service predicts stress.
- * Stress result card displayed with colour-coded level and message.
- * Weekly schedule auto-regenerated based on the new stress level.

Key Files:

- * frontend/src/pages/user/LifestylePage.jsx
- * frontend/src/pages/user/WellnessPage.jsx

2. Model Training

What it does: Trains a RandomForestClassifier on stress_dataset.csv to predict stress level from the 8 lifestyle features.

Training Pipeline

```
stress_dataset.csv
v load_data() -- split into X (8 features) and y (stress_level)
v SMOTE -- balance minority stress classes with synthetic samples
v train_test_split -- 80% training, 20% testing
v RandomForestClassifier(n_estimators=200, max_depth=5, n_jobs=-1)
v clf.fit(X_train, y_train) -- model learns patterns
v accuracy_score on test set
v cross_val_score -- confirm reliability across folds
v stress_model.pkl saved -- binary serialised model
v model_meta.json saved -- features, labels, accuracy
```

RandomForestClassifier

- * Builds 200 independent decision trees on random data/feature subsets.
- * Each tree votes on the stress class for a given input.
- * The majority vote determines the final prediction.
- * max_depth=5 prevents overfitting; n_jobs=-1 uses all CPU cores.

SMOTE

If "Very High" stress samples number only 5 while "Very Low" samples number 100, the model becomes biased toward predicting "Very Low". SMOTE generates synthetic "Very High" rows by interpolating between existing ones, equalising class representation and ensuring the model learns genuine patterns for all five stress levels.

Key Files:

- * ml-service/train.py
- * datasets/stress_dataset.csv
- * ml-service/model/stress_model.pkl

3. Algorithm Testing & Optimization

What it does: Serves the trained model as a live HTTP API (FastAPI) and continuously improves it through automatic retraining on newly confirmed user data.

API Endpoints

Endpoint	Method	Purpose
<code>/predict`</code>	POST	Predict stress level from 8 lifestyle values
<code>/add-data`</code>	POST	Append a confirmed data row to the dataset
<code>/retrain`</code>	POST	Manually trigger model retraining
<code>/health`</code>	GET	Service liveness check
<code>/model-info`</code>	GET	Return model accuracy and feature list

Prediction Process

- * Backend sends 8 lifestyle values to POST `/predict`.
- * Pydantic validates each value against its allowed range.
- * Values are reshaped into a 1×8 NumPy matrix.
- * `RandomForestClassifier.predict()` -- 200 trees vote, majority class wins.
- * `predict_proba()` -- averaged probability distribution across all trees.
- * Response: `{ "stress_level": 2, "stress_label": "Normal", "probabilities": {...} }`.

Live Optimization (Continuous Learning)

- * User confirms their stress level -> `/add-data` appends to `stress_dataset.csv`.
- * `RETRAIN_BATCH = 1` -- every new row triggers an automatic background retrain.
- * `subprocess.run(train.py)` -- retrains the full model without restarting the API.
- * Thread lock (`_model_lock`) ensures the old model continues serving predictions during retraining -- zero downtime.
- * `ml-service/app.py`

4. Scheduling

What it does: Automatically generates a personalised Mon-Sun weekly schedule based on the user's ML-predicted stress level and profile preferences.

Personalisation Factors

User Factor	Effect on Schedule
Stress Very High (4)	45-min work blocks, 20-min breaks, Gentle Yoga only
Stress High (3)	75-min work blocks, 15-min breaks, Light Walk
Stress Normal/Low/Very Low (0-2)	90-min work blocks, 10-min breaks, full exercise
Wake Time	Schedule anchors to custom time (default 07:00)
Work Hours	Work blocks placed within configured window
Workout Preference	HIIT / Moderate / Light / None
Meal Preference	Plant-Based / Keto / High-Protein / Mediterranean labels
Registered Holidays/Leave	Day replaced with "Rest Day / Holiday" slot
Custom Working Days	Non-working days treated as weekends

Slot Types in Every Generated Day

Slot Type	Description
Sleep	00:00 -> wake time, and bedtime -> 23:59
Morning Routine	Post-wake routine (20-30 min based on stress)
Meal	Breakfast / Lunch / Dinner (personalised labels)
Work	Focus blocks with short breaks; 2-4 blocks per day
Break	Short breaks between work blocks; mindful break at high stress
Exercise	Intensity matched to stress level and user preference
Leisure	Evening leisure activity (user's free-time preference)
Wind Down	Pre-bed routine (30-45 min based on stress)

Key Files:

- * backend/src/main/java/.../service/WeeklyScheduleService.java
- * backend/src/main/java/.../repository/ScheduleRepository.java

Complete System Architecture

React Frontend
LifestylePage WellnessPage SchedulePage

REST API (JWT auth)

Spring Boot Backend
UserController AuthService
WeeklyScheduleService WellnessService
MongoDB Atlas (users, schedules, wellness)

HTTP (internal)

FastAPI ML Service (app.py)
RandomForestClassifier (stress_model.pkl)
/predict /add-data /retrain

Trains on

stress_dataset.csv
(train.py -- offline or auto-retrain)

Key Viva Questions and Answers

Q: How does LifeMate predict stress level?

A: Eight daily lifestyle values (mood, workload, sleep, energy, social interaction, exercise, screen time, water) are fed to a RandomForestClassifier trained on labelled historical data. The 200-tree ensemble votes on one of five stress classes (Very Low -> Very High).

Q: Why was RandomForestClassifier chosen?

A: It trains in parallel (`n_jobs=-1`), is naturally resistant to overfitting, requires minimal hyperparameter tuning, and provides probability outputs (`predict_proba`) for all five classes -- enabling confidence visualisation.

Q: What is SMOTE and why is it needed?

A: The stress dataset has unequal class sizes -- "Very High" stress is rarer than "Normal". Without balancing, the model biases toward common classes. SMOTE generates synthetic minority-class samples by interpolating between existing ones, forcing the model to learn all five classes equally.

Q: How is the schedule personalised?

A: `WeeklyScheduleService.buildDay()` reads the user's stress index and profile (wake time, work hours, dietary and workout preferences, registered leave days) and varies work-block length, break duration, exercise type, and meal labels accordingly.

Q: What is continuous/live learning?

A: When a user confirms their stress label, /add-data appends the row to the CSV. Because RETRAIN_BATCH = 1, train.py is rerun automatically in the background after each new row. The updated model replaces the in-memory instance without API downtime -- the model improves with every confirmed user interaction.

Documentation -- LifeMate Project, Yasithzz, 2026